

CONSULTAR CON EXPRESIONES

__in: VERIFICA UN INTERVALO DE VALORES

```
def consultar5(request):  
    alumnos=Alumnos.objects.filter(nombre__in=["Juan", "Ana"])  
    return  
    render(request,"registros/consultas.html",{ 'alumnos':alumnos})
```

views.py

CONSULTAR CON EXPRESIONES

- Agregamos una nueva url:

```
path('consultas5', views_registros.consultar5,  
     name="Consulta5"),
```

urls.py

CONSULTAR CON EXPRESIONES

- Ejecutamos servidor y accedemos a la ruta:
 - `http://127.0.0.1:8000/consultas5`

Foto	Matricula	Nombre	Carrera	Turno
	UTM1234	Juan	TI	Matutino
	utm2	Ana	BIO	Vespertino
	UTM1234TIC	Juan	TI	Matutino

```
alumnos=Alumnos.objects.filter(nombre__in=["Juan", "Ana"])
```

CONSULTAR CON EXPRESIONES

__range : BUSQUEDA POR RANGO DE FECHAS

```
import datetime
```

```
def consultar6(request):  
    fechaInicio = datetime.date(2022, 7, 1)  
    fechaFin = datetime.date(2022, 7, 13)  
    alumnos=Alumnos.objects.filter(created__range=(fechaInicio,fechaFin))  
    return render(request,"registros/consultas.html",{ 'alumnos':alumnos})
```

views.py

CONSULTAR CON EXPRESIONES

- Agregamos una nueva url:

```
path('consultas6', views_registros.consultar6,  
     name="Consulta6"),
```

urls.py

CONSULTAR CON EXPRESIONES

Para visualizar la fecha se coloco el campo created en la lista de campos a mostrar en el archivo admin.py

```
list_display = ('matricula', 'nombre', 'carrera', 'turno', 'created')
```

NOMBRE	CARRERA	TURNO	CREATED
Monica	TI	Vespertino	July 15, 2021, 7:52 a.m.
Juan	TI	Matutino	July 15, 2021, 6:19 a.m.
Ana	BIO	Vespertino	July 1, 2021, 7:39 p.m.
Juan	TI	Matutino	June 25, 2021, 2:28 a.m.

Datos del
administrador

Consulta a realizar sobre los datos

```
fechaInicio = datetime.date(2021, 7, 1)
fechaFin = datetime.date(2021, 7, 13)
alumnos=Alumnos.objects.filter(created__range=(fechaInicio,fechaFin))
```

CONSULTAR CON EXPRESIONES

- Ejecutamos servidor y accedemos a la ruta:
 - `http://127.0.0.1:8000/consultas6`

Salida

Foto	Matricula	Nombre	Carrera	Turno
	UTM7785TI	Monica	TI	Vespertino
	UTM1234	Juan	TI	Matutino
	utm2	Ana	BIO	Vespertino

```
fechaInicio = datetime.date(2021, 7, 1)
fechaFin = datetime.date(2021, 7, 13)
alumnos=Alumnos.objects.filter(created__range=(fechaInicio,fechaFin))
```

CONSULTAR ENTRE MODELOS - RELACIONES

Si recuerdas, nuestro modelo de Alumnos está relacionado con un modelo comentario que administramos desde el módulo de admin:

MÓDULOS	
Alumnos	+ Add
Comentarios	+ Add
Comentarios Contactos	+ Add

Search

◀ 2021 June 29

Action: Go 0 of 1 selected

☐	CLAVE	COMENTARIO
☐	1	No Inscrito

1 Comentario

CONSULTAR ENTRE MODELOS - RELACIONES

- Para realizar una consulta entre modelos, empleamos la función filter, pero en la condición es posible indicar el nombre del modelo y el campo que deseamos condicionar.
- En nuestro ejemplo la condición es que el alumno este en comentario y que el comentario sea No Inscrito :

```
def consultar7(request):  
    #Consultando entre modelos  
    alumnos=Alumnos.objects.filter(comentario__coment__contains='No inscrito')  
    return render(request,"registros/consultas.html',{'alumnos':alumnos})
```

views.py

CONSULTAR ENTRE MODELOS - RELACIONES

- Agregamos una nueva url:

```
path('consultas7', views_registros.consultar7,  
     name="Consulta7"),
```

urls.py

CONSULTAR ENTRE MODELOS - RELACIONES

- Ejecutamos servidor y accedemos a la ruta:
 - <http://127.0.0.1:8000/consultas7>

Salida

Foto	Matricula	Nombre	Carrera	Turno
	UTM1234TIC	Juan	TI	Matutino

```
alumnos=Alumnos.objects.filter(comentario__coment__contains='No Inscrito')
```



CONSULTAS DIRECTAS CON SQL



CONSULTAS DIRECTAS CON SQL

- Django permite realizar dos formas de consultas SQL:
- **ORM** (Object Relational Mapper): Permite interactuar con nuestra base de datos sin la necesidad de conocer SQL.
- Usando **Manage.raw()**: **raw()** es un método del administrador que se puede utilizar para realizar consultas SQL sin procesar que devuelven instancias de modelo, permite ejecutar SQL personalizado y directo.

CONSULTAS DIRECTAS CON SQL

```
def consultasSQL(request):  
    alumnos=Alumnos.objects.raw('SELECT  
matricula,nombre, carrera, turno, imagen FROM  
registros_alumnos WHERE carrera="TI" ORDER BY  
turno DESC')  
  
    return  
render(request,"registros/consultas.html",  
{ 'alumnos':alumnos})
```

views.py

CONSULTAS DIRECTAS CON SQL

- Agregamos una nueva url:

```
path('consultasSQL', views_registros.consult  
asSQL, name="sql"),
```

urls.py

- Ejecutamos servidor y accedemos a la ruta:
 - <http://127.0.0.1:8000/consultasSQL>

CONSULTAS DIRECTAS CON SQL

FieldDoesNotExist at /consultasSQL

Raw query must include the primary key

Request Method: GET

Request URL: http://127.0.0.1:8000/consultasSQL

Django Version: 3.2.4

Exception Type: FieldDoesNotExist

Exception Value: Raw query must include the primary key

Exception Location: /Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/site-packages/django/db/models/query.py, line 1499, in iterator

Python Executable: /Library/Frameworks/Python.framework/Versions/3.9/bin/python3

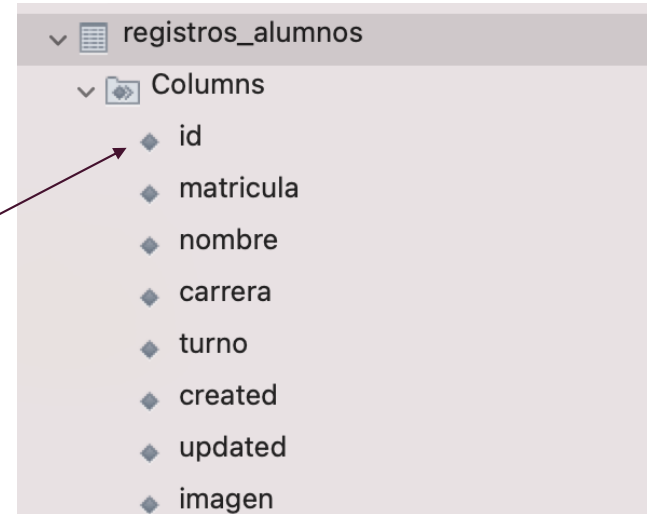
Python Version: 3.9.5

Python Path: ['/Users/elena/Documents/proyectos Django/prueba',
'/Library/Frameworks/Python.framework/Versions/3.9/lib/python39.zip',
'/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9',
'/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/lib-dynload',
'/Library/Frameworks/Python.framework/Versions/3.9/lib/python3.9/site-packages']

Server time: Thu, 22 Jul 2021 01:25:10 +0000

CONSULTAS DIRECTAS CON SQL

En este mecanismo de consulta, tiene la restricción de consultar el campo `id` ya que como identificador se toma como requerido para procesos que puedan ser requeridos en la aplicación. En nuestro caso el modelo de alumno no cuenta con llave primaria, sin embargo si accedemos a la estructura de la tabla observarán que fue agregado automáticamente.



A screenshot of a database interface showing the structure of a table named 'registros_alumnos'. The table is expanded, showing a list of columns. The first column, 'id', is highlighted with a red arrow pointing from the text in the paragraph to its left. The columns are: id, matricula, nombre, carrera, turno, created, updated, and imagen.

registros_alumnos
Columns
id
matricula
nombre
carrera
turno
created
updated
imagen

CONSULTAS DIRECTAS CON SQL

```
def consultasSQL(request):  
    alumnos=Alumnos.objects.raw('SELECT id, ← Agrega  
matricula,nombre, carrera, turno, imagen FROM  
registros_alumnos WHERE carrera="TI" ORDER BY  
turno DESC')  
  
    return  
    render(request,"registros/consultas.html",  
{'alumnos':alumnos})
```

views.py

CONSULTAS DIRECTAS CON SQL

- Ejecutamos servidor y accedemos a la ruta:
 - <http://127.0.0.1:8000/consultasSQL>

Foto	Matricula	Nombre	Carrera	Turno
	UTM7785TI	Monica	TI	Vespertino
	UTM1234TIC	Juan	TI	Matutino
	UTM1234	Juan	TI	Matutino

```
SELECT id,matricula,nombre,carrera,turno,imagen FROM registros_alumnos WHERE  
carrera="TI" ORDER BY turno DESC
```



Explore the ORM before using raw SQL!

The Django ORM provides many tools to express queries without writing raw SQL. For example:

- The [QuerySet API](#) is extensive.
- You can [annotate](#) and [aggregate](#) using many built-in [database functions](#). Beyond those, you can create [custom query expressions](#).

Before using raw SQL, explore [the ORM](#). Ask on one of [the support channels](#) to see if the ORM supports your use case.



Warning

You should be very careful whenever you write raw SQL. Every time you use it, you should properly escape any parameters that the user can control by using [params](#) in order to protect against SQL injection attacks. Please read more about [SQL injection protection](#).

ORM vs SQL

PARTICIPACIÓN

ORM

SQL

- VENTAJAS Y DESVENTAJAS
- ¿Por qué DJANGO apuesta por ORM?